



Módulo 3 Fundamentos de animación 2D

Preparado por: Ing. Samuel Jiménez



Unidad 3

Menú y transformaciones en OpenGL

Objetivos Generales

- ***Conocerla estructura de un programa para la creación de menús desplegables.***
- ***Explicar las funciones para la definición, ubicación y orientación de la cámara que utiliza OpenGL***



Menú Desplegables

- La librería Glut incorpora un conjunto de instrucciones que ayudan de una manera fácil a que se añadan menús desplegables en una aplicación.

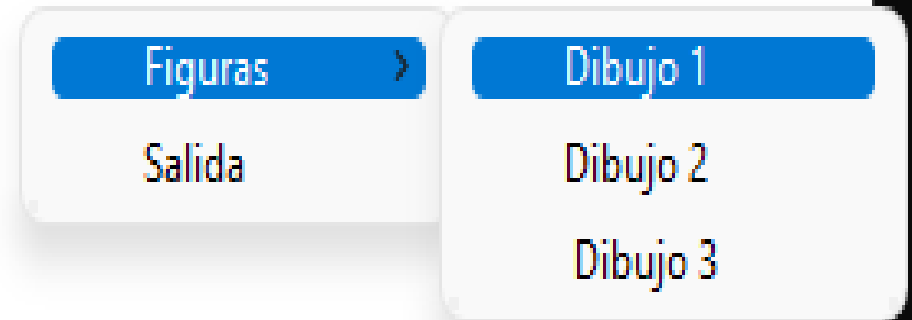
Int glutCreateMenu (void (*función) (int valor));

La función glutCreateMenu devuelve un valor entero que identifica de forma univoca el menú que se esta creando.

El parámetro función especifica el nombre de la función que se debe invocar cuando una opción del menú sea seleccionada.

Ejemplo

```
int sub1=glutCreateMenu (Menu);  
glutAddMenuEntry("Dibujo 1", 1);  
glutAddMenuEntry("Dibujo 2", 2);  
glutAddMenuEntry(" Dibujo 3", 3);  
glutCreateMenu (Menu);  
glutAddSubMenu("Figuras", sub1);  
glutAddMenuEntry("Salida", 4);  
glutAttachMenu(GLUT_RIGHT_BUTTON);  
glutDisplayFunc(displaydefault);  
glutKeyboardFunc(keyInput);  
glutSpecialFunc(specialKeyInput);  
glutMainLoop();
```



Menú Desplegables

La función que asigna el menú actual a un botón concreto del ratón es:

void glutAttachMenu (intbotón)

El menú se hace visible cuando el botón es presionado y el cursor permanece dentro de la ventana de la aplicación.

Los valores del parámetro botón puede ser:

- GLUT_LEFT_BUTTON = Botón izquierdo
- GLUT_RIGHT_BUTTON = Botón derecho
- GLUT_MIDDLE_BUTTON = Botón central

Ejemplo

```
int sub1=glutCreateMenu (Menu);  
glutAddMenuEntry("Dibujo 1", 1);  
glutAddMenuEntry("Dibujo 2", 2);  
glutAddMenuEntry(" Dibujo 3", 3);  
glutCreateMenu (Menu);  
glutAddSubMenu("Figuras", sub1);  
glutAddMenuEntry("Salida", 4);  
glutAttachMenu(GLUT_RIGHT_BUTTON);  
glutDisplayFunc (displaydefault);  
glutKeyboardFunc (keyInput);  
glutSpecialFunc (specialKeyInput);  
glutMainLoop();
```



Menú Desplegables

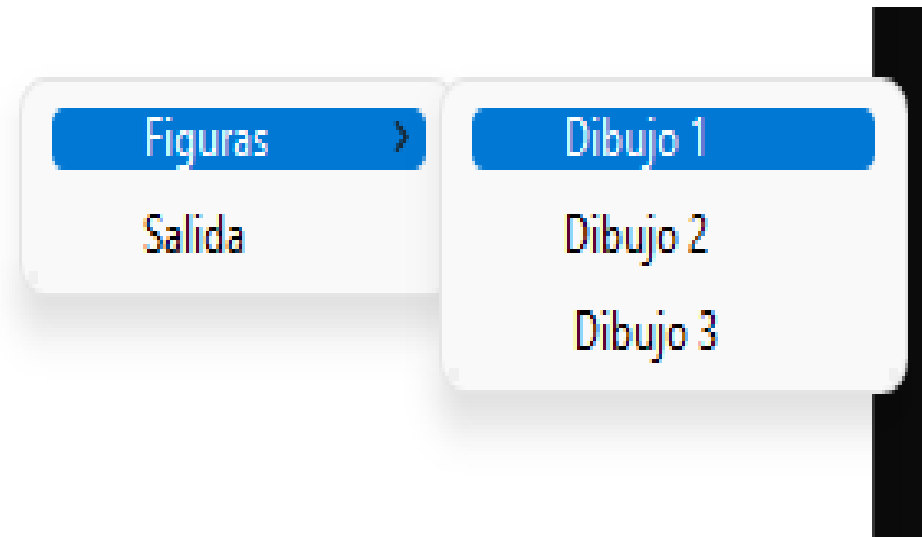
La función que añade una opción al menú:

void glutAddMenuEntry(char*nombre, int valor);

El parámetro nombre se añade al menú actual y se identifica por el parámetro valor, que es utilizado para distinguir una opción de otras.

Ejemplo

```
int sub1=glutCreateMenu (Menu);  
glutAddMenuEntry("Dibujo 1", 1);  
glutAddMenuEntry("Dibujo 2", 2);  
glutAddMenuEntry(" Dibujo 3", 3);  
glutCreateMenu (Menu);  
glutAddSubMenu("Figuras", sub1);  
glutAddMenuEntry("Salida", 4);  
glutAttachMenu(GLUT_RIGHT_BUTTON);  
glutDisplayFunc(displaydefault);  
glutKeyboardFunc(keyInput);  
glutSpecialFunc(specialKeyInput);  
glutMainLoop();
```



Menú Desplegables

La función:

void glutAddSubMenu(char*nombre,intmenú);

Adicional a la creación de un menú como un conjunto de opciones, se da la posibilidad de que una opción del menú a su vez sea menú, lo que se denomina submenú.

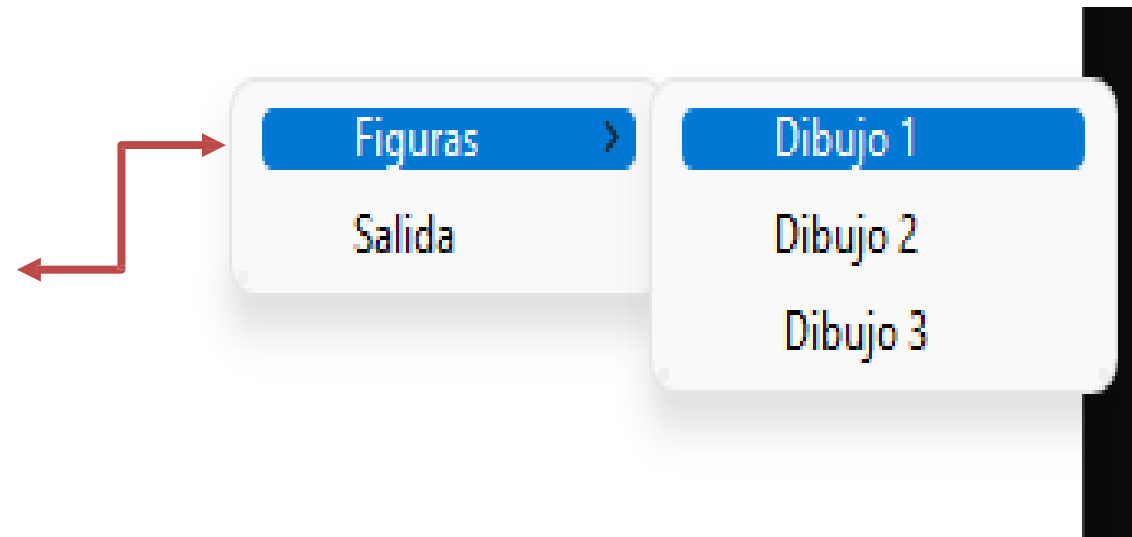
La función:

void glutSetMenu (int menú)

El parámetro menú es el identificador del menú que se desea designar como menú actual.

Ejemplo

```
int sub1=glutCreateMenu (Menu);  
glutAddMenuEntry("Dibujo 1", 1);  
glutAddMenuEntry("Dibujo 2", 2);  
glutAddMenuEntry(" Dibujo 3", 3);  
glutCreateMenu (Menu);  
glutAddSubMenu("Figuras", sub1);  
glutAddMenuEntry("Salida", 4);  
glutAttachMenu(GLUT_RIGHT_BUTTON);  
glutDisplayFunc(displaydefault);  
glutKeyboardFunc(keyInput);  
glutSpecialFunc(specialKeyInput);  
glutMainLoop();
```



Ejemplo

```
#include <gl/glut.h>

float alpha, beta;
int x0, y0;

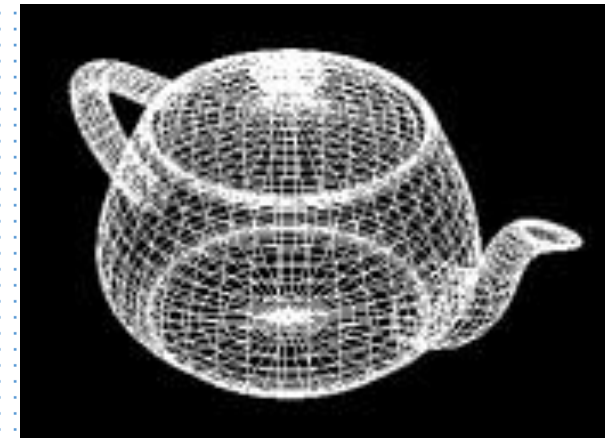
void display(void) {
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(20.0f, 1.0f, 1.0f, 10.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0.0f, 0.0f, 5.0f,
              0.0f, 0.0f, 0.0f,
              0.0f, 1.0f, 0.0f);
    glRotatef(alpha, 1.0f, 0.0f, 0.0f);
    glRotatef(beta, 0.0f, 1.0f, 0.0f);
    glutWireTeapot(0.5);
    glFlush();
    glutSwapBuffers();
}

void onMouse(int button, int state, int x, int y) {
    if ( (button == GLUT_LEFT_BUTTON) & (state == GLUT_DOWN) ) {
        x0 = x; y0 = y;
    }
}

void onMotion(int x, int y) {
    alpha = (alpha + (y - y0));
    beta = (beta + (x - x0));
    x0 = x; y0 = y;
    glutPostRedisplay();
}

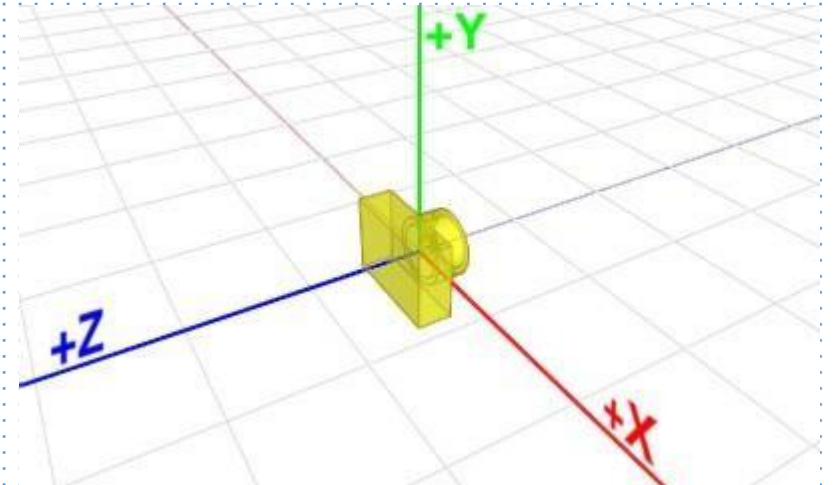
void main(void) {
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(400, 400);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Ejemplo de menus");
    glutDisplayFunc(display);
    glutMouseFunc(onMouse);
    glutMotionFunc(onMotion);
    glutMainLoop();
}
```

El siguiente ejemplo muestra una tetera que podremos rotar utilizando el mouse. Mientras que el mouse esté pulsado, según como lo movamos, moveremos la tetera. Prestad especial atención a las funciones `onMouse` y `onMotion`, especialmente a la obtención de los ángulos de rotación. Si la cámara estuviera situada en otro sitio, la obtención de los ángulos tendría que variar.



Cámara en OpenGL

- Los parámetros que definen la cámara sintética son:
 - ✓ La posición donde se sitúa el ojo del fotógrafo
 - ✓ El tamaño donde se toma la fotografía
 - ✓ La orientación de la cámara
 - ✓ El objetivo que se utiliza. Es decir, la amplitud del campo de visión
 - ✓ Distancia de la cámara



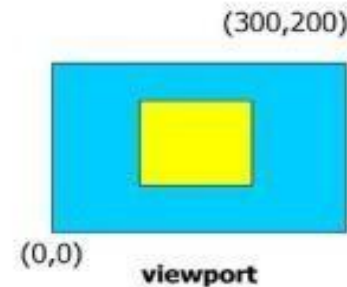
Cámara en OpenGL

- Si se requiere hacer uso de la cámara en una aplicación con OpenGL se debe determinar el tamaño de la fotografía y la encargada de realizar este funcionamiento es:

`void glViewport(0,0 anchoVentana, altoVentana)`

```
DrawQuad()
{glViewport(0,0,300,200);
 glMatrixMode(GL_PROJECTION);
 glLoadIdentity();
 glOrtho2D(-1,1,-1,1);
 glBegin(GL_QUADS);
 glColor3f(1,1,0);

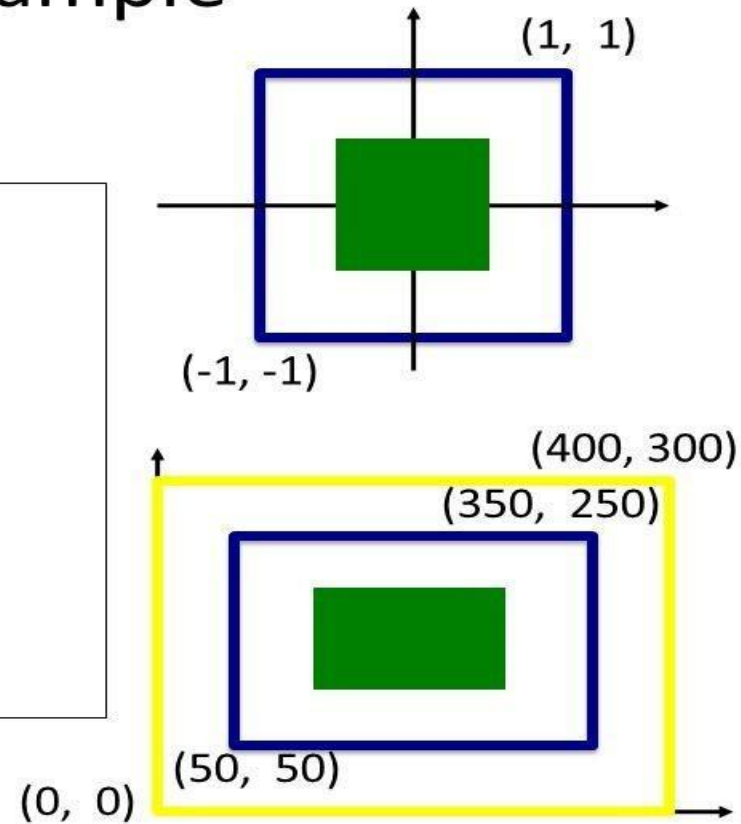
 glVertex2i(-0.5,-0.5);
 glVertex2i(+0.5,-0.5);
 glVertex2i(+0.5,+0.5);
 glVertex2i(-0.5,+0.5);
 glEnd();
```



Ejemplo de Sistema de Coordenadas en OpenGL

An Example

```
void DrawQuad()
{
    glViewport(50, 50, 300, 200);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);
    glBegin(GL_QUADS);
    glVertex2f(-0.5, -0.5);
    glVertex2f( 0.5, -0.5);
    glVertex2f( 0.5,  0.5);
    glVertex2f(-0.5,  0.5);
    glEnd();
}
```



Cámara en OpenGL

Luego de que se haya elegido una determinada cámara, objetivo y película, se debe situar la cámara para hacer la fotografía, para ello debemos definir los tamaños y las formas tanto de la foto como del campo de visión.

Dentro de OpenGL se encuentra la orden `gluLookAt` que es la encargada de situar la cámara en la escena.

La función `glutLookAt` permite establecer la posición de la cámara, el punto de interés de la escena al que se mira y su inclinación.

Formato:

`void glutLookAt(Gldouble ojox, Gldouble ojoy, Gldouble ojoz, Gldouble pInteresx, Gldouble pInteresy, Gldouble pInteresz, Gldouble veticalx, Gldouble veticalY, Gldouble veticalz)`

Cámara en OpenGL

Formato:

void glutLookAt(Gldouble ojox, Gldouble ojoy, Gldouble ojoz, Gldouble pInteresx, Gldouble pInteresy, Gldouble pInteresz, Gldouble veticalx, Gldouble verticalY, Gldouble verticalz)

Definiciones del formato:

ojo: es el punto donde se debe situar la cámara o el ojo del observador

Pinteres: es el punto e interés hacia el que se mira y que estará en el centro de la fotografía

Vertical: es un vector fijado a la cámara y que apunta hacia arriba.

Ejemplo de glutLookAt

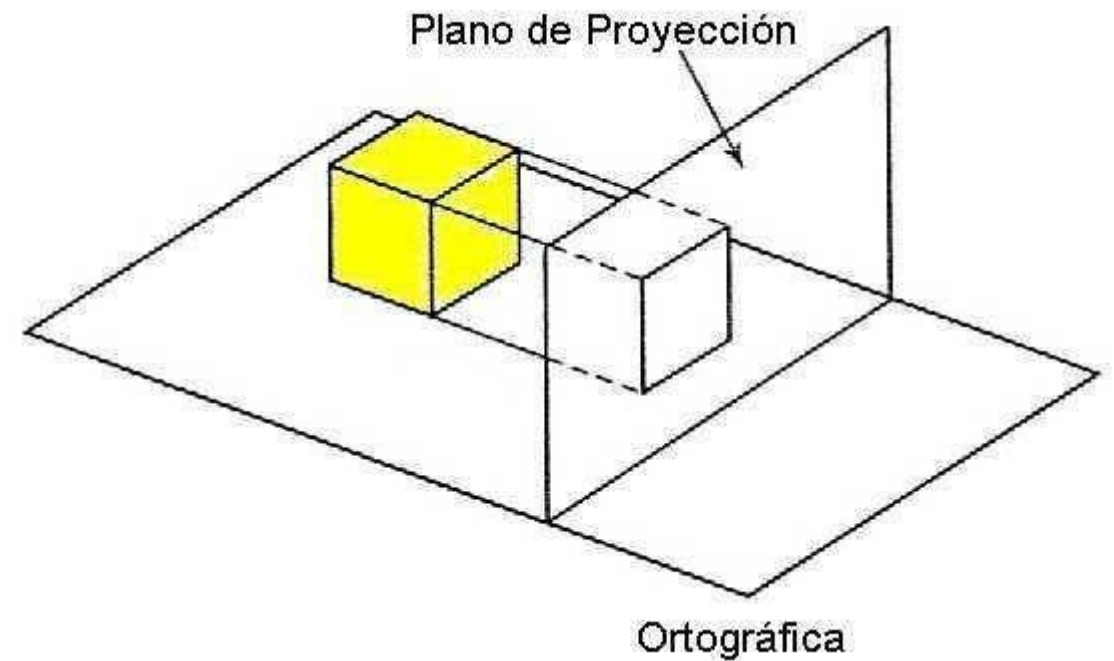
```
#include <gl/glut.h>

float aspect = 1.0f;
void display(void) {
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(20.0f, aspect, 1.0f, 10.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0.0f, 0.0f, 5.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f);
    glutWireTeapot(0.5);
    glFlush();
    glutSwapBuffers();
}
void onSize(int sx, int sy) {
    glViewport(0, 0, sx, sy);
    aspect = (float) sx / (float) sy;
}
void main(void) {
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(400, 400);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Ejemplo de menus");
    glutDisplayFunc(display);
    glutReshapeFunc(onSize);
    glutMainLoop();
}
```

Proyecciones en OpenGL

Estas proyecciones son:

- ☐ Proyección paralela u ortográfica
- ☐ Proyección perspectiva



Visualización en paralela

Cuando se especifica una proyección paralela, la orden de OpenGL que define el campo de visión de la cámara es:

Void glOrtho(Gldouble izquierdaCV, Gldouble derechaCV, Gldouble abajoCV, Gldouble arribaCV, Gldouble delanteCV, Gldouble detrásCV)

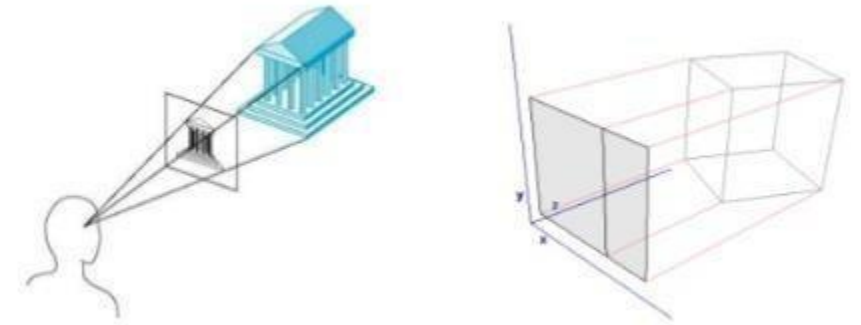
Visualización en perspectiva

Cuando se especifica una proyección perspectiva, la función de OpenGL que define el campo de visión de la cámara es:

Void glFrustum (Gldouble izquierdaCV, Gldouble derechaCV, Gldouble abajoCV, Gldouble arribaCV, Gldouble delanteCV, Gldouble detrasCV)

Asimismo, podemos usar la función de la librería de utilidades:

Void gluPerspective (Gldouble anguloY, Gldouble relacionXaY, Gldouble delanteCV, Gldouble detrásCV)



anguloY: es el ángulo vertical en grados que tienen el campo de visión.

relacionXaY: es la relación de aspecto entre el ancho y el alto del negativo donde se hace la foto.

DelanteCV, detrasCV: cumplen la misma función limitadora que en la función glFrustum

Ejemplo de perspectiva

```
glMatrixMode(GL_PROJECTION);  
  
glPushMatrix(); /* Guardamos la matriz de proyeccion */  
glLoadIdentity(); /* P = I */  
gluPerspective(30.0f, anchura / altura, 1.0, 10.0);  
  
glMatrixMode(GL_MODELVIEW);  
  
glPushMatrix(); /* Guardamos la matriz del modelo */  
glLoadIdentity(); /* M = I; */  
  
/* Colocamos la camara en el punto deseado */  
gluLookat(0, 5, 5, 0, 0, 0, 0, 1, 0);  
  
/* Dibujamos una tetera roja centrada en (-1, 0, 0) */  
glTranslatef(-1.0f, 0.0f, 0.0f);  
glColor3f(1.0f, 0.0f, 0.0f);  
glutWireTeapot(0.5f);  
  
glPopMatrix(); /* Recuperamos la antigua matriz del modelo */  
  
glMatrixMode(GL_PROJECTION);  
glPopMatrix(); /* Recuperamos la antigua matriz de proyeccion */
```

Transformaciones en OpenGL

- Las operaciones básicas de OpenGL que permite situar los objetos en el espacio, es decir, aplicarles transformaciones son:
 - ✓ glTranslate
 - ✓ glRotate
 - ✓ glScale

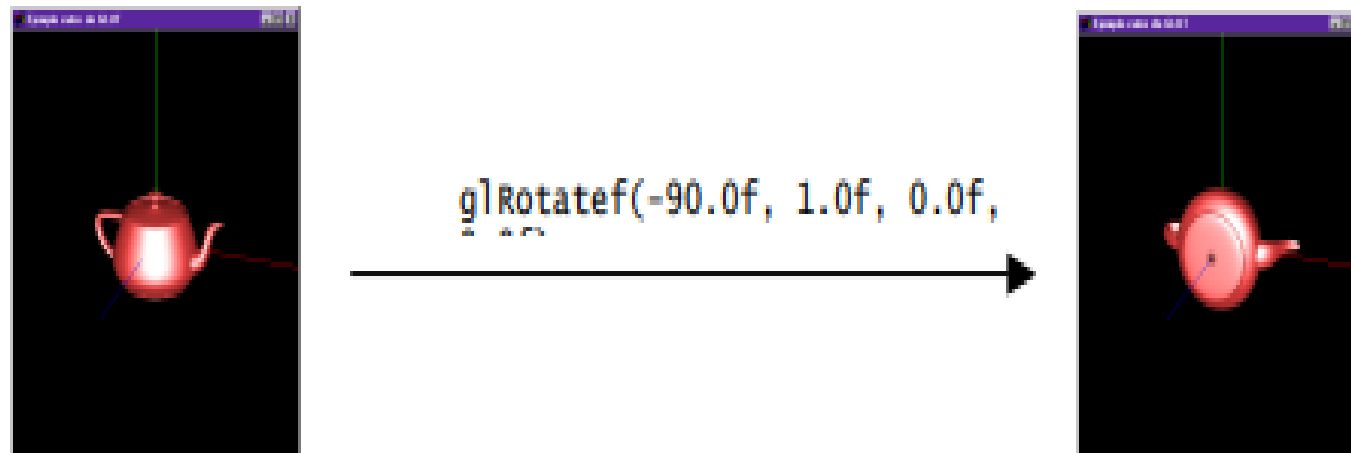
OJO: En OpenGL es muy importante el orden en el que vayamos a realizar las transformaciones. No es lo mismo trasladar algo y después rotarlo, que primero rotarlo y luego trasladarlo.

Transformaciones en OpenGL

Ejemplo:

Rotación

Se define mediante la primitiva `glRotatef(alpha, x, y, z)`. Esta función multiplica la matriz actual por una matriz de rotación de α grados respecto al eje (x, y, z) .

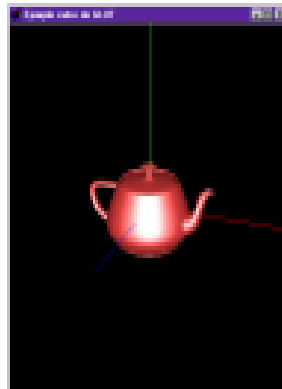


Transformaciones en OpenGL

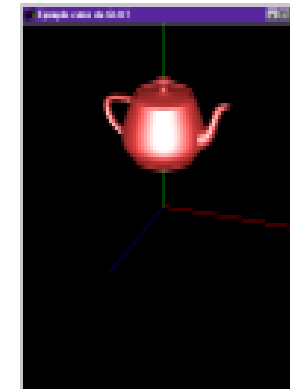
Ejemplo:

Translación

Se define mediante la primitiva `glTranslatef(x, y, z)` Aplica una translación en (x, y, z) sobre la matriz actual.



`glTranslatef(0.0f, 0.5f, 0.0f)`

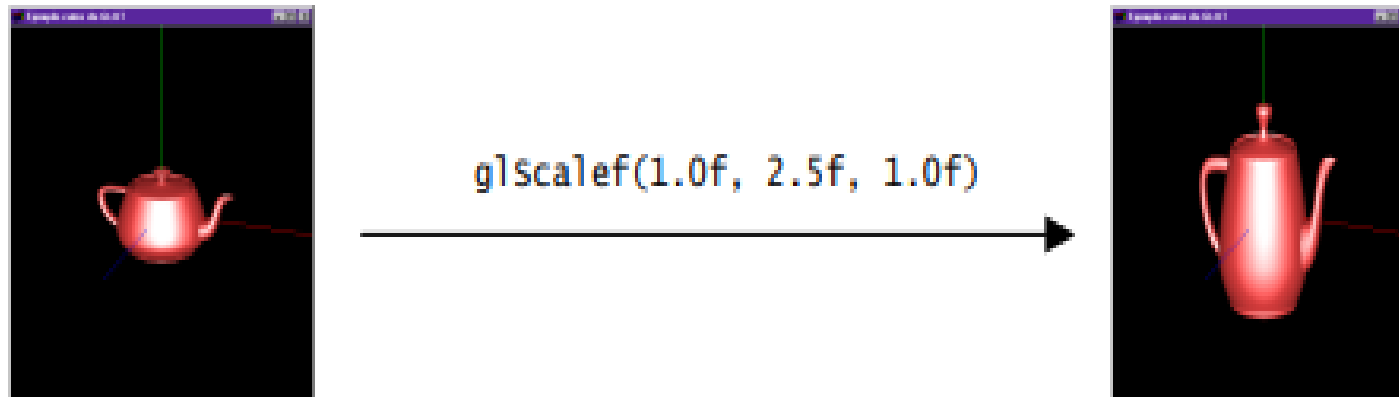


Transformaciones en OpenGL

Ejemplo:

Escalado

Se define mediante la primitiva `glScalef(sx, sy, sz)` Escalado de cada uno de los ejes.



Transformaciones en OpenGL

Ejemplo:

ROTATE + TRANSLATE

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
for (i=0; i<10; i++) {  
    glPushMatrix();  
    glRotatef(i * 360 / 10, 0.0f, 1.0f, 0.0f);  
    glTranslatef(0.5f, 0.0f, 0.0f);  
    glutSolidTeapot(0.25);  
    glPopMatrix();  
}
```



Transformaciones en OpenGL

Ejemplo:

TRANSLATE + ROTATE

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
for (i=0; i<10; i++) {  
    glPushMatrix();  
    glTranslatef(0.5f, 0.0f, 0.0f);  
    glRotatef(i * 360 / 10, 0.0f, 1.0f, 0.0f);  
    glutSolidTeapot(0.25);  
    glPopMatrix();  
}
```



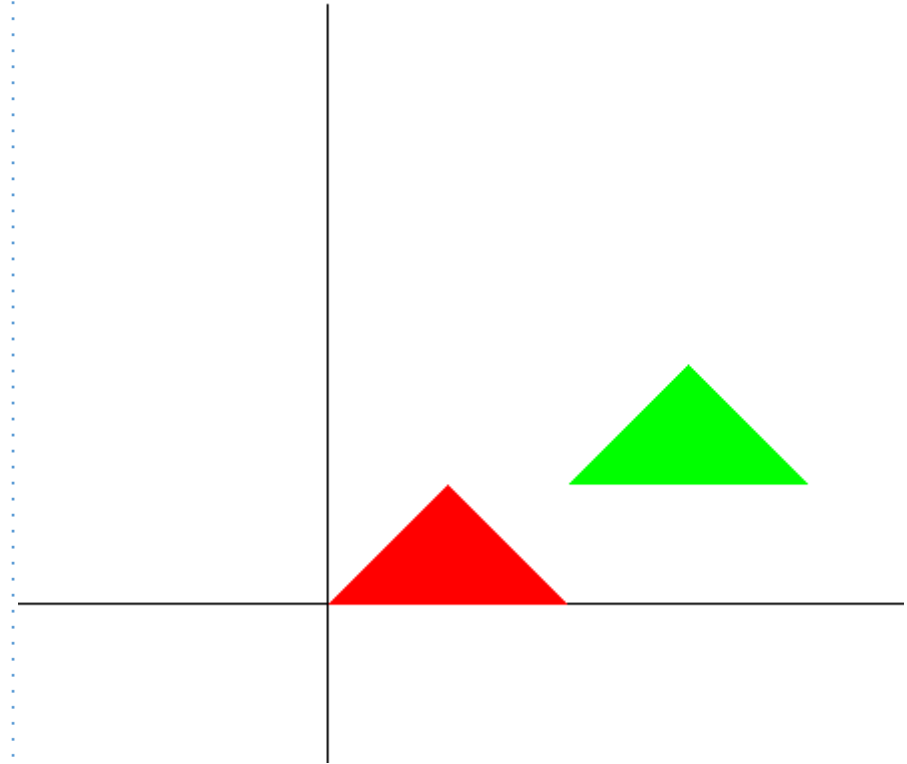
Transformaciones en OpenGL

Ejemplo:

```
glBegin(GL_TRIANGLES);
glColor3f(1,0,0);
glVertex2f(.2f,.2f);
glVertex2f(0.0f,0.0f);
    glVertex2f(0.4f,0.0f);
glEnd();

glTranslated(0.4,0.2,0);

glBegin(GL_TRIANGLES);
glColor3f(0,1,0);
glVertex2f(.2f,.2f);
glVertex2f(0.0f,0.0f);
    glVertex2f(0.4f,0.0f);
glEnd();
glLoadIdentity();
glFlush();
```



¡GRACIAS!

